



WiFit : étude du projet WiMANS et réalisation d'un dataset à partir d'une solution à bas coût

LU3IN400 - Projet de recherche

Jules Perrin de Brichambaut

Tifenn Fabrici-Lourenco

Nadir Mohamed Jalaludeen Akbar Marécar

Beeverly Gourdet

Rapport final

Janvier 2025 - Mai 2025

Sommaire

Résumé	3
1 Introduction	4
1.1 Capacités d'utilisation en fonction du milieu	4
1.2 CSI et RSSI	4
1.3 Organisation du projet	4
2 Le Projet WiMANS	5
2.1 WiMANS	5
2.1.1 Présentation	5
2.1.2 Structure de la base de données WiMANS	5
2.1.3 Modèles pour l'évaluation du dataset WiMANS	6
2.2 Visualisation et comparaison des amplitudes des signaux CSI	7
3 Entraînement et Comparaison des modèles	9
3.1 Mise en oeuvre de l'apprentissage	9
3.2 Problématique : l'overfitting ou le sur-apprentissage	9
3.3 Performances sur les précisions des tâches	10
3.3.1 Choix du modèle à utiliser	12
4 Travail Expérimental	13
4.1 Utilisation d'un Raspberry Pi	13
4.1.1 Problématiques matérielles et logiciels	13
4.1.2 Nexmon & capture CSI	13
4.2 Utilisation d'un ESP32	14
4.2.1 Collecte des données CSI	14
4.3 Construction de notre dataset	15
4.3.1 Organisation de la collecte	15
4.4 Entraînement d'un Réseau de Neurones à Convolution (CNN)	17
5 Conclusion du projet	18
6 Annexes	19
6.1 Installation et utilisation de Nexmon et Nexmon CSI	19
6.1.1 Installation du Raspberry Pi 4 et modification du firmware	19
6.1.2 Configuration de l'interface d'écoute et utilisation	20
6.2 Transformation des logs ESP32 en amplitudes .npy (NumPy)	21
6.3 Implémentation du modèle ResNet[10]	22
6.4 Algorithme de prédiction en temps réel	23
6.5 Administratif et outils	25
6.6 Réunions	25
Bibliographie	26

Résumé

L'utilisation d'onde wifi et de l'intelligence artificielle pour localiser des humains ou des objets est une approche économique et non-intrusive en plein essor. En particulier, l'analyse des signaux wifi permet de détecter des individus ou des objets en étudiant les variations de ces ondes.

Notre projet de recherche s'est articulé autour du jeu de données WiMANS[11], contenant plus de 9 heures d'enregistrement de signaux wifi avec leurs mesures CSI (*Channel State Information*) dans une multitude de situations distinctes, et incluant plusieurs modèles d'intelligence artificielle afin d'évaluer leurs qualités.

Dans un premier temps, nous avons observé l'impact de notre activité dans l'espace sur les mesures CSI afin de mieux comprendre cette technologie. Nous avons ainsi remarqué des différences assez nettes, montrant que ces signaux sont bien affectés par ces facteurs environnementaux.

Nous avons ensuite comparé 3 modèles du dataset (THAT, CLSTM, CNN-2D) sur leurs architectures, leurs précisions, leurs temps d'apprentissage, ainsi que le temps de prédiction selon la taille du batch. Malgré des problèmes d'overfitting, nous avons pu déterminer que le réseau de neurones à convolution CNN-2D a un temps d'apprentissage et de prédiction rapide, tout en ayant la meilleure précision pour identifier les individus.

Par la suite, en s'aidant d'un micro-contrôleur ESP32 pour la collecte de mesures CSI, nous avons construit notre propre jeu de données de 1000 échantillons de 3 secondes pour le comptage d'individus (allant de 0 à 4), obtenu en fragmentant des enregistrements plus longs. Nous avons ainsi pu entraîner un réseau neuronal résiduel ResNet[10] sur ces données et obtenu une précision de 61.89 %, démontrant ainsi le potentiel de cette technologie pour la détection humaine.

1 Introduction

La reconnaissance d'objets et la détection de mouvements humains font naturellement penser à l'exploitation de données visuelles telles que les vidéos ou les images. Des bibliothèques comme Detectron2 ou Mmaction2 favorisent ces formats pour entraîner leurs modèles de détection. Toutefois, les données vidéos sont soumises aux mêmes contraintes que l'oeil humain : la faible luminosité ou les obstacles tels que des murs ou des meubles volumineux peuvent rendre difficile l'identification des objets ou des mouvements et limiter l'efficacité des modèles basés sur une analyse visuelle. C'est pourquoi une analyse avec des signaux wifi est une approche prometteuse. En effet, ceux-ci ont la capacité de pouvoir traverser certains matériaux et obstacles permettant, en observant des variations dans les signaux perçus, d'identifier un objet. Cela aussi bien en termes de localisation dans l'espace que de reconnaissance de la nature de l'objet. Le projet WiMANS[11] est un exemple d'utilisation des mesures CSI pour trois tâches principales : la détection de l'identité d'un ou plusieurs individus avec les modèles, leur position dans la salle ainsi que la reconnaissance des mouvements effectués simultanément. Nous cherchons à évaluer et adapter l'implémentation des modèles proposés par le projet WiMANS[11] en utilisant une solution à bas coût en terme de matériel et de temps de calcul tout en assurant une précision sur les tâches satisfaisantes.

1.1 Capacités d'utilisation en fonction du milieu

En fonction du milieu, les ondes wifi peuvent évoluer différemment. Dans le cadre d'un environnement complexe avec de multiples objets dans une pièce, et donc une vision non-directe entre l'émetteur et le récepteur, les ondes peuvent se réfléchir sur différents supports, voire en fonction des matériaux, s'atténuer très fortement. L'organisation d'une pièce et le nombre d'objets présents seront donc des facteurs importants à prendre en compte. Des résultats significatifs pourraient n'être obtenables que dans des environnements précis.

Parallèlement, la présence de bruit par l'émission de multiples signaux wifi, notamment en environnement urbain, représente une problématique supplémentaire en impliquant un possible traitement supplémentaire.

Enfin, le matériel nécessaire, généralement coûteux pour la capture de mesures CSI est une contrainte limitante à l'obtention de données dans n'importe quel milieu.

1.2 CSI et RSSI

Le *Received Signal Strength Indicator* (RSSI) caractérise le niveau de puissance du signal reçu sur l'intégralité de la bande de fréquence utilisée. Celui-ci est implémenté au sein de la couche logicielle et est disponible avec l'intégralité des puces wifi aujourd'hui.

Le *Channel State Information* (CSI) appartient à la couche physique contrairement au RSSI. Ce faisant, celui-ci nécessite des puces wifi compatibles afin de pouvoir collecter les informations plus détaillées. Parmi ces informations, on obtient ainsi le détail précis sur les différents *channels* utilisés sur la plage de fréquence. Cela permet d'obtenir un état de la propagation du signal dans l'espace avec une précision bien plus fine.

Longtemps utilisé dans le cadre d'application de localisation d'objets dans l'espace, le RSSI est aujourd'hui largement remplacé par le CSI lorsque cela est possible, de par la précision supplémentaire obtenue, permettant de nombreuses nouvelles applications, mais également en supprimant la nécessité de l'utilisation de multiples émetteurs ou récepteurs lorsque l'objet à localiser n'est pas unique ou que l'environnement est complexe.

1.3 Organisation du projet

Le projet est découpé en trois parties principales :

- Dans une première partie, étudier et analyser les mesures d'information sur l'état du canal (CSI) provenant du jeu de données WiMANS[11].
- Ensuite, comparer et évaluer les performances des modèles d'apprentissage automatique mis à disposition pour la détection d'objets et la reconnaissance de l'activité humaine en fonction de la quantité de données.
- Étudier la capacité à capturer les mesures CSI avec un Raspberry Pi ou un ESP32, construire notre jeu de données et à mettre en place un modèle d'apprentissage automatique pour le comptage d'individus avec les outils à notre disposition.

Les différents codes utilisés pour ce projet sont disponibles sur le dépôt GitLab suivant:
<https://gitlab.sorbonne-universite.fr/Nadir/l3-projet-wimans>

2 Le Projet WiMANS

2.1 WiMANS

2.1.1 Présentation

WiMANS[11] est le premier ensemble de données de détection d'activité multi-utilisateurs basé sur le wifi avec des mesures *Channel State Information* (CSI). WiMANS[11] contient 11286 échantillons CSI de deux bandes wifi (2,4 / 5 GHz) et des vidéos synchronisées pour des tâches de référence et inexplorées (par exemple, l'estimation de la pose multi-utilisateurs). Chaque échantillon de 3 secondes comprend de 0 à 5 utilisateurs effectuant simultanément des activités identiques/différentes, annotées avec les identités, les emplacements et les activités des utilisateurs (anonymisés).

Enregistrant plus d'informations que les précédents ensembles de données existant, cela nous permet d'expérimenter des modèles avec un potentiel intéressant, notamment pour la localisation et l'identification d'objets. Ce jeu de données ne considère que des pièces sans obstacles majeures et ne contient pas de captures de mesures à travers des murs par exemple.

2.1.2 Structure de la base de données WiMANS

Le dataset WiMANS[11] est composé de plusieurs types de fichiers :

annotation.csv : Ce fichier décrit chaque échantillon de la base de données par plusieurs caractéristiques :

- **label** : Le label d'un échantillon prend la forme `act_<group>_<sample>` où `group` représente l'indice du groupe de personnes participant dans l'échantillon et `<sample>` l'indice de échantillon dans le groupe
- **environment** : Les différentes salles où ont lieu l'enregistrement des données. Les salles sont `classroom`, `meeting_room` et `empty_room`
- **wifi_band** : La bande de fréquence wifi utilisée 2.4 GHz ou 5 GHz
- **number_of_users** : Le nombre de personnes participant à l'échantillon
- **user_i_location** : Position de la personne à l'indice `i` dans la salle. Les positions sont `"a"`, `"b"`, `"c"`, `"d"`, `"e"`. La position est non définie si la personne ne fait pas partie du groupe.
- **user_i_activity** : Activité de la personne à l'indice `i` dans la salle. Les actions possibles sont `"nothing"`, `"walk"`, `"rotation"`, `"jump"`, `"wave"`, `"lie_down"`, `"pick_up"`, `"sit_down"`, `"stand_up"`. Comme la position, peut-être non définie si la personne ne fait pas partie du groupe.

Les fichiers .npy : Les fichiers `.npy` permettent de sauvegarder des tableaux Numpy sous forme binaire. Ce format permet de récupérer le tableau avec la méthode `np.load()` tout en préservant la dimension originale. Les tableaux contenus dans les fichiers `*.npy` représentent l'amplitude des instances CSI sous forme de nombres complexes pour chaque échantillon. Les matrices sont de dimensions 3000 x 3 x 3 x 30.

Les fichiers .mat : Les fichiers `*.mat` sont des fichiers binaires Matlab et permettent de récupérer les variables d'un environnement de travail. La restauration avec `scio.loadmat()` retourne un dictionnaire de forme `nom_variable : matrices`

2.1.3 Modèles pour l'évaluation du dataset WiMANS

Les concepteurs du dataset WiMANS[11] ont fourni plusieurs modèles afin de l'évaluer sur les tâches de "location", "identity", "activity".

Chaque modèle est écrit avec Pytorch et se trouve dans un fichier <nom_model>.py

De plus, on y trouve deux types de modèles, ceux pour le wifi CSI, et ceux pour les exemples vidéos.

Plus de détails sont donnés dans le papier WiMANS [11, p. 10] en ce qui concerne leurs performances respectives.

Tâches

On distingue trois tâches que ces modèles peuvent effectuer pour de l'inférence :

- La reconnaissance d'activité humaine (activity) dont le but est de deviner ce que font les personnes, comme par exemple sauter, se baisser, ou bouger le bras.
- La localisation des personnes (location), quant à elle, permet d'inférer la position d'une personne dans la pièce.
- Puis enfin, il y a l'identification des personnes (identify) dont le but est d'identifier une personne précise.

Modèles CSI

Pour les modèles CSI, on y trouve plusieurs types de modèles mis à disposition:

- Plusieurs variantes de modèles CNN (convolutional neural network) [14, 20]
- Plusieurs variantes de modèles LSTM (long short-term memory) qui est un type de RNN (recurrent neural network) [25]
- Une variante de random forest, ST-RF (Random Forest based on Short-time Fourier Transform) [25]
- Un modèle THAT (two-stream convolution augmented transformer for human activity recognition) [12]
- Et enfin, un modèle MLP classique (Multi-layer perceptron) [22]

L'utilisation de modèles CNNs[14, 20] est une première approche intéressante de par la grande dimension des données CSI, ainsi que leurs capacités à pouvoir extraire des caractéristiques significatives de leurs matrices d'amplitude.

Cela est particulièrement utile pour la détection d'individu ou d'objet qui vont venir créer des variations dans les signaux reçus et ainsi se faire détecter.

Quant aux modèles LSTM[25], leur atout principal est le fait de pouvoir retenir des informations, cela en fait de bons candidats pour la reconnaissance d'activité humaine où l'interprétation de cette même activité va dépendre des actions précédentes de la personne.

La présence d'un modèle MLP[22] est intéressante, étant donné qu'il a été l'un des modèles les plus utilisés et les plus populaires, bien que de nos jours, il ait été un peu dépassé par d'autres modèles.

2.2 Visualisation et comparaison des amplitudes des signaux CSI

Afin de représenter les données CSI, on souhaite afficher les amplitudes des signaux à chaque instant. Initialement, les matrices d'amplitude ont pour valeur des nombres complexes. Après transformation de la matrice en calculant pour chaque nombre complexe son module, on obtient un tableau d'amplitudes traitées à chaque instant de l'échantillon. Les amplitudes et leurs intensités sont représentées à l'aide de la colormap `viridis`.

On souhaite étudier les différences d'intensités des amplitudes lorsque l'on cible et modifie une unique caractéristique dans différents échantillons similaires. Il faut en premier lieu identifier dans la base de données de plus de 10 000 échantillons, les échantillons comparables avec une seule caractéristique différente et dans lesquels tous les autres attributs sont similaires.

Pour cela, on convertit le fichier CSV `annotations.csv` en `DataFrame`. On peut alors extraire les échantillons dont le seul attribut qui varie est l'activité grâce à la fonction `amplitude_solo_activity`, la salle avec `amplitude_solo_room` ou encore selon le nombre de personnes à l'aide de `amplitude_multiple`.

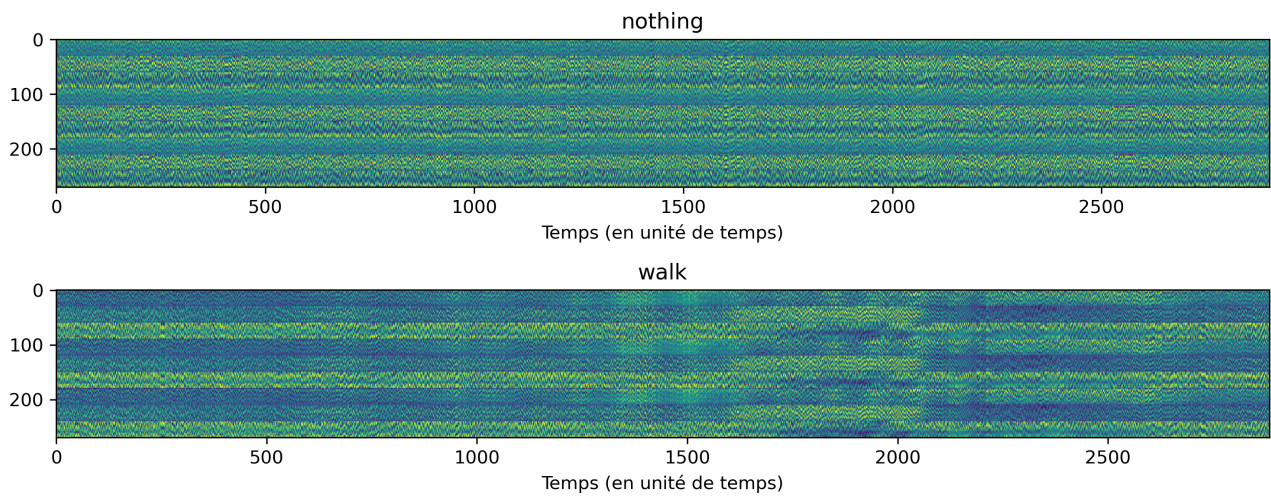


Figure 1: Comparaison des amplitudes des mesures CSI lorsque l'individu 1 marche (**walk**) ou reste immobile (**nothing**). L'individu est situé à l'emplacement (a) dans la salle `classroom` avec une bande wifi 2.4 GHz

Dans la **Figure 1**, les abscisses représentent le temps (en unités de temps), tandis que les ordonnées correspondent aux sous-porteuses des signaux wifi. Chaque échantillon dure 3 secondes et contient 3000 unités de temps, avec un envoi de 1000 paquets par seconde. On remarque qu'au cours du temps l'amplitude des signaux est moins intense lorsque l'individu marche que lorsqu'il ne fait rien. Cette différence d'intensité d'amplitudes peut être expliquée par les interférences causées par les mouvements sur la propagation des signaux wifi dans la salle. En effet, les mouvements modifient les trajets des signaux entre les transmetteurs et receveurs pendant la durée de l'action. Les mouvements ne sont pas le seul facteur influençant l'intensité des signaux. D'autres attributs, tels que la configuration de la salle et le nombre de personnes présentes peuvent affecter la manière dont les signaux se propagent.

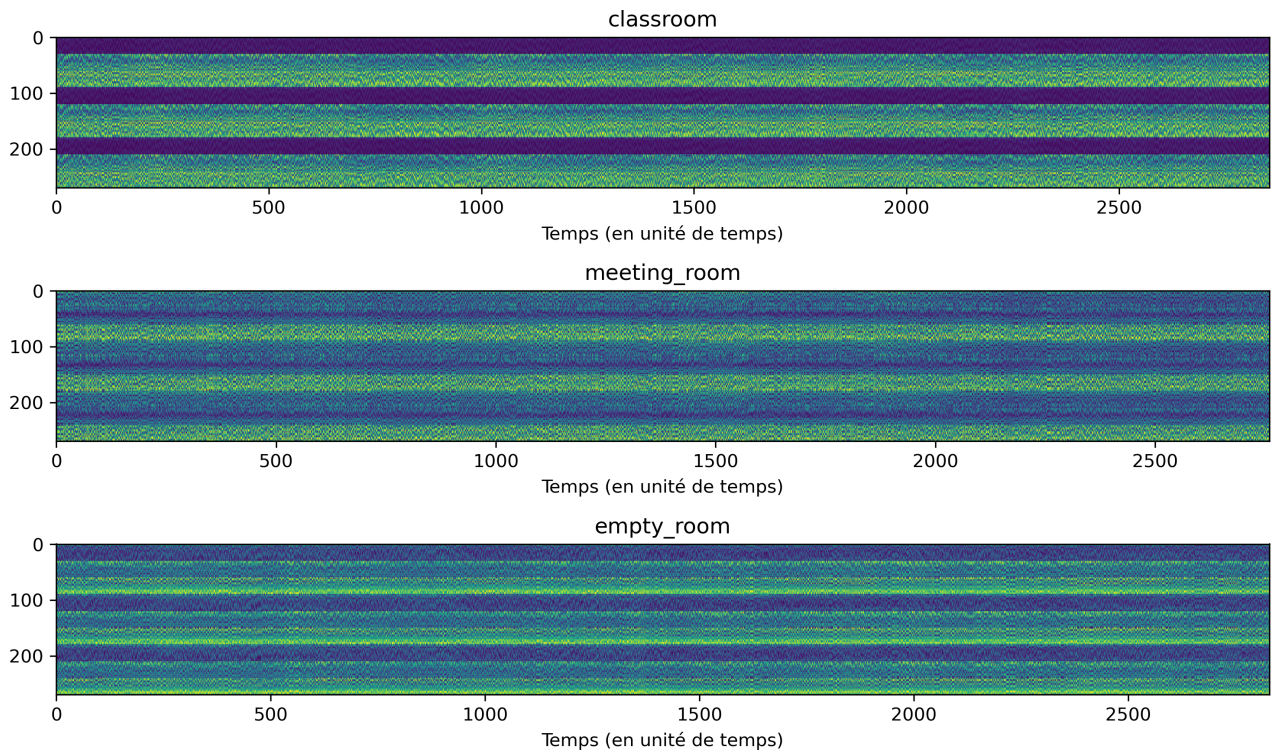


Figure 2: Comparaison des amplitudes des mesures CSI lorsque l'individu 1 est situé dans une salle vide **empty_room**, dans une salle de réunion **meeting_room** et une salle de classe **classroom**. L'individu saute **jump** et est situé à l'emplacement (a) avec une bande wifi 2.4 GHz

Dans la **Figure 2**, on observe plusieurs bandes sombres dans la salle **classroom** indiquant que certaines zones sont bien moins traversées par les signaux que d'autres avec des amplitudes fortes (bandes vertes). La présence de mobiliers, comme plusieurs bureaux et chaises, peut amener les signaux à emprunter d'autres chemins plus directs et moins obstrués créant ainsi des zones de forte intensité. La salle **meeting_room** possède également des bandes sombres moins intenses que la salle de classe, car la salle de réunion possède un agencement plus aéré de la pièce avec les tables placées en carré. On identifie également les bandes de sous-porteuses les plus sollicitées avec cependant moins d'intensité d'amplitudes. Enfin, la salle vide **empty_room** possède également des chemins moins empruntés, mais contrairement aux deux autres salles, les zones des chemins les plus empruntées sont plus fines : il existe donc moins de chemins directs et la propagation est plus homogène.

3 Entraînement et Comparaison des modèles

Les modèles dont nous disposons sont très lourds et nécessitent beaucoup de temps et d'énergie afin de les entraîner. Il a été nécessaire de faire des choix pour ne pas s'éparpiller.

Ainsi, nous avons choisi de nous restreindre aux 3 modèles suivants : CNN-2D[14, 20], CLSTM[13], THAT[12].

Table 1: Comparaison des avantages et inconvénients des modèles CNN-2D, CLSTM et THAT

Modèle	Procédé	Avantages	Inconvénients
CLSTM [13]	- Possède une mémoire qui permet de conserver les informations sur le long terme qui permet de mettre en évidence les dépendances entre les événements passés et actuels comme les LSTM classiques - En entrée, prends en plus un contexte supplémentaire.	- Plus efficaces que les LSTM classiques pour détecter les dépendances à long terme. - Meilleures performances dans la gestion des corrélations spatio-temporelles pour la prédiction de données lié à l'environnement, comme les mesures CSI.	- Demande plus de mémoire - Difficulté supplémentaire avec le contexte fournit qui ne doit pas induire d'erreur.
CNN-2D [14, 20]	- Analyse les représentations par image (2D) des amplitudes CSI avec des filtres afin de reconnaître d'extraire des motifs, comme les variations de signaux.	- Capable de reconnaître les motifs automatiquement	- A besoin d'un jeu de donnée conséquent : sensible à l'overfitting.
THAT [12]	- Utilise deux flux pour analyser à la fois les variations dans le temps et entre les canaux, puis applique un transformeur amélioré pour reconnaître différents types de motifs.	- Permet d'apprendre les dépendances à long terme entre les événements passés et actuels comme le modèle CLSTM.	- Temps de calcul conséquent.

Ces modèles sont assez différents dans leur architecture et font office d'une bonne représentation de l'ensemble des modèles fournis avec le dataset dû au fait que les autres sont principalement des variations du CNN[14, 20] ou du LSTM[25]. Le ST-RF[25] avait aussi initialement été considéré, mais dû à certains bugs lors de son apprentissage, il a été écarté.

L'approche que nous avons choisi d'étudier est la variation de la quantité de données sur la performance des modèles dans le but d'identifier un seuil minimal pour obtenir des résultats significatifs.

Cela en faisant varier la taille de la base d'apprentissage, de 1GB à 32GB, puis sur le dataset entier.

3.1 Mise en oeuvre de l'apprentissage

Dans un premier temps, il a fallu partitionner le jeu de données en plusieurs échantillons de tailles allant de 1GB à 32GB à l'aide d'un script python respectant la proportion des classes (fragmentation.py).

Afin d'exploiter notre nouvelle partition, nous avons ensuite écrit un script Bash (*run_model.sh*) permettant de lancer l'apprentissage des différents modèles sur toutes les tailles du jeu de données.

Il a de plus été nécessaire de modifier le code original fourni avec WiMANS[11] afin de ne lancer que les modèles que nous avons sélectionnés ainsi que la sauvegarde de leurs poids et d'une prise en compte de la taille de l'échantillon.

Les résultats de chaque modèle sont ensuite enregistrés dans le dossier *result*.

3.2 Problématique : l'overfitting ou le sur-apprentissage

On remarque dans la **Figure 3** que pour les trois modèles, les précisions pour les trois tâches sont anormalement hautes pour les tailles d'échantillons de 1GB et 2GB: le nombre d'échantillons étant trop faibles, les modèles peuvent mémoriser les exemples d'entraînement ce qui gonfle les précisions finales. À partir des échantillons de 4GB, on observe une croissance dans les précisions qui nous indiquent que les modèles apprennent correctement et que les performances s'améliorent avec l'augmentation de la taille des données fournies.

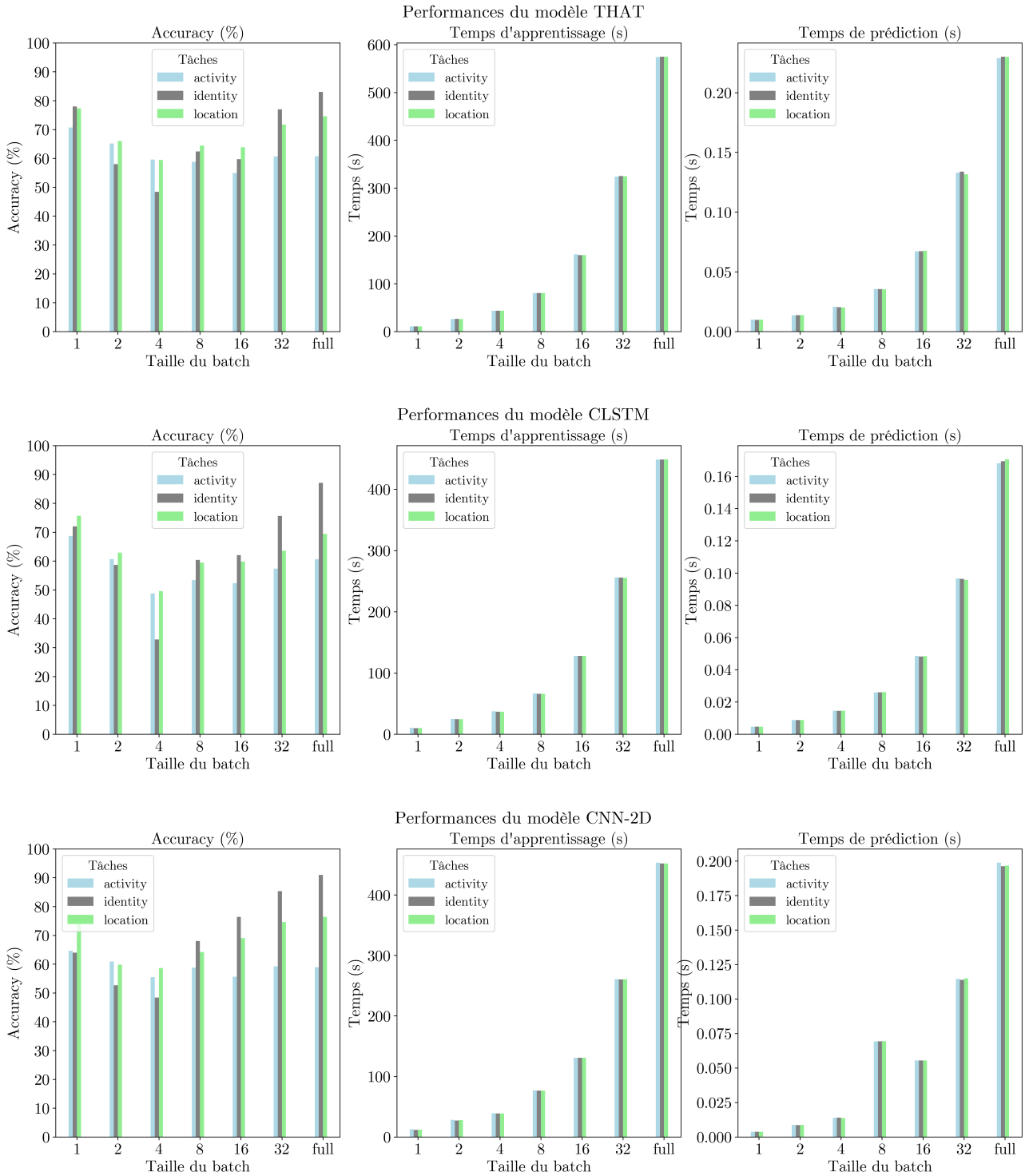


Figure 3: Performances des modèles CNN_2D, CLSTM et THAT sur la reconnaissance des tâches, **identity**, **localisation** et **activity** utilisant les bandes dual wifi 2.4GHz et avec 5 utilisateurs.

3.3 Performances sur les précisions des tâches

On s'intéresse à trouver une taille d'échantillon efficace pour allier vitesse et réduction des temps d'apprentissage et de prédiction tout en gardant la meilleure précision possible. Pour cela, on se penche premièrement les tâches les plus ou moins réussies par les modèles. La reconnaissance **identity** est la plus réussie par les modèles, suivie de **location** et enfin **activity**. Pour tous les modèles, l'augmentation de la taille des données à chaque tranche de taille de batch favorisent la reconnaissance des utilisateurs. Pour les modèles CLSTM[13] et THAT[12], les passages d'une taille d'échantillon à un autre jouent faiblement sur la reconnaissance de la localisation contrairement au modèle CNN_2D[14, 20] qui profite d'une nette amélioration. Enfin, pour tous les modèles, la

reconnaissance de l'activité reste le maillon faible qui évolue peu comparée aux deux autres tâches. On remarque que CNN_2D a une moins bonne précision sur les activités comparée aux autres modèles.

On aimerait également déterminer si prendre une taille d'échantillon inférieure nous permettrait de gagner un temps d'apprentissage et de prédiction significatif. Pour le modèle THAT[12] avec le temps d'apprentissage le plus élevé, réduire la taille de l'échantillon à 32GB a permis de presque réduire son temps d'entraînement par deux, idem pour son temps de prédiction. Lorsque l'on compare les précisions entre l'échantillon entier et réduit à 32GB, un choix intéressant selon le contexte d'utilisation serait de sacrifier des bonnes classifications pour gagner du temps lors des lancements. Cependant, descendre à 16GB serait perdre trop de précision sur la reconnaissance de l'utilisateur. Les raisonnements sont similaires pour les modèles CNN_2D[14, 20] et CLSTM[13], bien que le premier sera moins impacté par la baisse de précision sur *identity*. La réduction de l'échantillon pour CNN_2D lui permettrait de rivaliser avec la vitesse de CLSTM tout en ayant de meilleurs résultats sur la reconnaissance d'utilisateurs

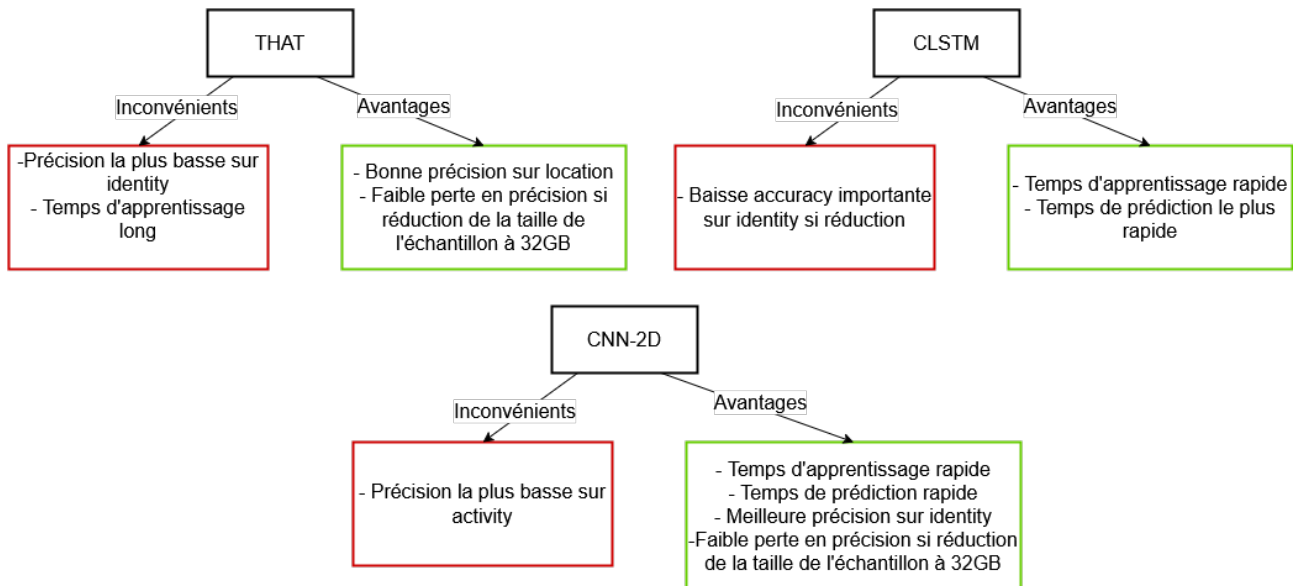


Figure 4: Diagramme représentant les avantages et inconvénients des modèles THAT, CLSTM et CNN-2D

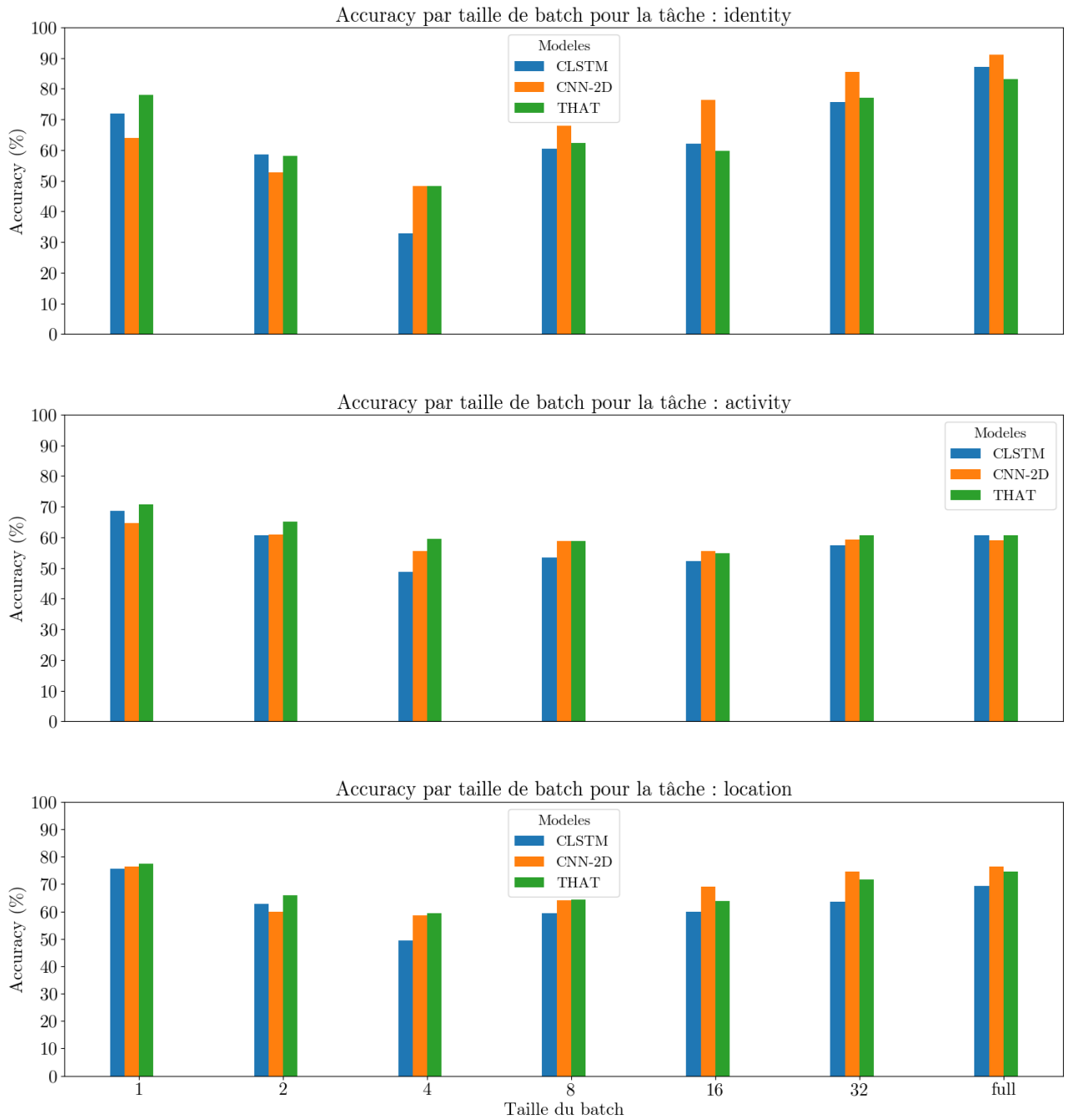


Figure 5: Comparaison des performances des modèles CNN_2D, CLSTM et THAT sur la reconnaissance des tâches, *identity*, *localisation* et *activity* utilisant une bande wifi 2.4GHz et avec 5 utilisateurs.

3.3.1 Choix du modèle à utiliser

Concernant les résultats, ils semblent en accord avec ceux du projet WIMANS[11] : les précisions des modèles sur la reconnaissance de l'activité sont environ de 60% pour les modèles, en deçà des autres activités qui vont entre 80% et 90% de précision en utilisant la bande wifi de 2.4GHz. Le modèle CNN-2D[14, 20] apparaît comme un choix judicieux : on pourrait réduire la taille d'échantillon en ne perdant pas de précisions trop importantes sur les tâches tout en bénéficiant d'un temps d'apprentissage et de tests rapides.

De plus, c'est un modèle moins complexe que les deux autres, le rendant plus simple à implémenter sur un Raspberry Pi ou une autre machine limitée pour de l'inférence.

4 Travail Expérimental

La problématique majeure à la capture de mesure CSI est l'utilisation d'un matériel permettant un accès à la couche PHY. Le développement de solution utilisant des micro-ordinateurs et cartes programmables à bas coût permet la démocratisation de ce type de mesure.

Nous explorerons ici 2 solutions. La première consiste en l'utilisation d'un Raspberry Pi et du projet nexmon_CSI [18] et la seconde exploite les micro-contrôleurs ESP32.

L'avantage principale du Raspberry Pi sera sa capacité de capture sur la plage des 5GHz mais le faible prix et la faible taille des ESP32 permettent un déploiement simple et multiple.

4.1 Utilisation d'un Raspberry Pi

4.1.1 Problématiques matérielles et logiciels

La problématique principale est la capacité de mise en place et d'utilisation de la puce wifi en mode moniteur. Dans ce contexte, la puce wifi bcm43455c0 utilisé sur les Raspberry Pi 4 est une bonne candidate.

Malheureusement, en l'absence de documentation récente, et sachant que le projet n'est plus mis à jour depuis plusieurs années et ne prend ainsi pas en charge les versions ultérieurs du kernel et du système d'exploitation, nous sommes obligés d'utiliser une version antérieure en forçant une utilisation de version antérieure.

Par ailleurs, les mises à jour du bootloader, et la modification de son fonctionnement et de ses dépendances, des cartes Raspberry Pi empêche l'utilisation d'une version suffisamment ancienne et force donc la nécessité de trouver un compromis pour exploiter le peu de documentation présente, souvent décrivant une version plus ancienne.

Enfin, l'instabilité induite par la modification du firmware, et en fonction de la version choisie, peu provoqué l'incapacité de collecte de données sur certaines fréquences et sous-canaux.

La procédure d'installation est décrite dans la sous-section de l'annexe 6.1.

4.1.2 Nexmon & capture CSI

Le passage de l'interface wifi en mode moniteur permet la capture de paquets transitant sur le sous-canal de la fréquence wifi utilisé.

Lors de la capture d'une frame correcte, nexmon CSI réalise le calcul et forge une trame contenant les informations de la mesure CSI pour la frame capturée. Ces données peuvent ensuite être récupéré à l'aide d'un tcpdump et exporter sous la forme d'un fichier *.pcap* pour une analyse dans Wireshark ou à travers différents script python pour visualiser et traiter les mesures.

Il est également possible d'exporter, à travers un tunnel UDP, les trames forgées sur un autre ordinateur pour permettre leur analyse direct.

Les trames UDP forgées ont pour adresse source 10.10.10.10 et 255.255.255.255 pour adresse de destination sur le port 5500. La structure de données intégrées est décrite dans le tableau 2 ci-dessous.

Bytes	Type	Name	Description
4	uint32	Magic Bytes	0x11111111
6	uint8[6]	MAC 1	Address 1 - recv
6	uint8[6]	MAC 2	Address 2 - tran
6	uint8[6]	MAC 3	Address 3
2	uint16	FC	Frame's Control field
2	uint16	SC	Frame's SC field
1	int8	RSSI	Instantaneous frame RSSI
10	uint16[5]	Reserved	Reserved for future use
2	uint16	CSS	Core and Spatial Stream
2	uint16	Chanspec	Chanspec used during extraction. See 'nexutil -k'.
2	uint16	Chip Version	Chip Version
variable	int16[]	CSI Data	Each CSI sample is 4 bytes with interleaved Int16 Real and Int16 Imaginary. There are 'bandwidth * 3.2' OFDM subcarriers per channel, and a CSI sample for every subcarrier is present.

Table 2: Structure de la trame UDP forgée par Nexmon

4.2 Utilisation d'un ESP32

Au vu des difficultés liées à la collecte des données CSI avec un Raspberry Pi 4, nous avons également exploré la possibilité de collecte avec un micro-contrôleur ESP32. Cette solution étant largement mieux documentée, nous avons pu avoir des résultats beaucoup plus facilement.

4.2.1 Collecte des données CSI

Les puces des ESP32 sont développées par Espressif Systems, ces derniers mettent directement à disposition un code sur leur GitHub permettant la collecte de données CSI[3].

Une fois le micro-contrôleur connecté à un ordinateur via câble USB, on peut directement téléverser le code fourni par Espressif sur l'ESP32 via ESP-IDF[6], leur logiciel de conception de projets pour ESP32.

Finalement, une fois ESP-IDF installée et lancée, la procédure complète se résume aux 5 commandes suivantes :

```
#Télécharger le projet
git clone https://github.com/espressif/esp-csi/

#Se placer dans le bon répertoire
cd ./esp-csi/examples/get-started/csi\_recv\_router

#Sélectionner la puce du micro-contrôleur
idf.py set-target esp32

#Configurer le point d'accès auquel on veut se connecter
idf.py menuconfig

#Transférer le code et écouter le trafic
idf.py flash monitor
```

Par la suite, une fois le transfert du code terminé, différentes informations devraient s'afficher dans le terminal. Appuyer sur T puis L tout en restant appuyé sur Ctrl devrait commencer l'enregistrement des différentes lignes du terminal, dans un fichier logs .txt dans le dossier où on se situe.

Chaque ligne se présente selon la structure :

type, id, mac, rssi, rate, sig_mode, mcs, bandwidth, smoothing, not_sounding, aggregation, stbc, fec_coding, sgi, noise_floor, ampdu_cnt, channel, secondary_channel, local_timestamp, ant, sig_len, rx_state, len, first_word, data

Dans notre cas, seul le dernier élément, data, nous importe, étant donné que c'est ici que se situe l'information du CSI.

Elle se présente sous la forme d'une liste $[x_1, x_2, \dots, x_{128}]$, alternant la partie imaginaire puis la partie réelle de chaque nombre complexe composant la mesure CSI.

On peut ainsi, avec du parsing, se retrouver avec les mesures CSI nécessaires pour la suite. Le code utilisé afin de rendre ces informations exploitables pour nos objectifs est disponible dans la sous-section de l'annexe 6.2.

Après quelques calculs, nous avons remarqué qu'on enregistrerait 50 signaux par seconde, contre 1000 pour le dataset WiMANS[11]. Néanmoins, cela devrait être suffisant pour avoir des résultats, étant donné que certains datasets ont un taux encore plus faible tout en ayant des résultats significatifs[9].

De plus, la méthode qu'on a utilisée conseille d'utiliser des antennes externes, mais cette méthode se faisant avec de la soudure et des antennes qu'on n'a pas de disponibles, nous allons omettre l'utilisation de ces derniers.

4.3 Construction de notre dataset

4.3.1 Organisation de la collecte

Afin d'avoir assez de données pour l'entraînement d'un modèle, nous avons pour objectif d'avoir plus de 1000 échantillons, d'une durée de 3 secondes.

Étant donné que l'on souhaite compter le nombre d'individus, et qu'on est 4, il faut donc des échantillons avec 0, 1, 2, 3, ainsi que 4 individus. Nous allons tenter d'avoir au minimum 200 échantillons de chaque classe, en variant les membres à chaque fois afin d'également enregistrer l'identité de chaque personne qui passe.

En nous identifiant par les lettres A, B, C et D :

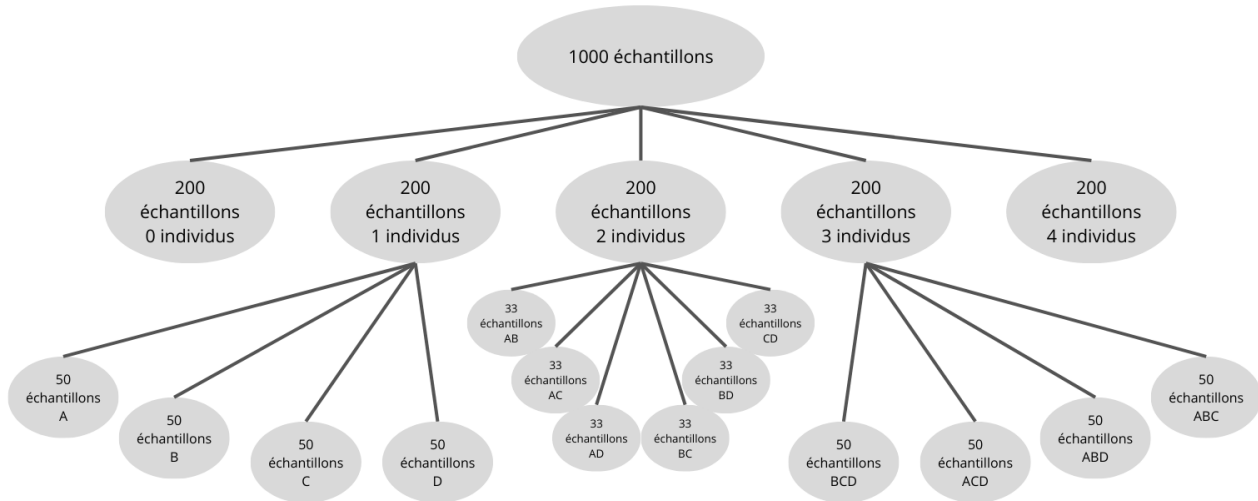


Figure 6: Répartition des différents échantillons que l'on souhaite récolter

On s'est placé dans le box BDL47 de la Bibliothèque des Licences de Sorbonne Université afin de collecter les données CSI, en s'aidant d'un smartphone Xiaomi Mi5 en tant que point d'accès wifi.



Figure 7: L'ESP32 envoyant les mesures CSI à un ordinateur ainsi que le Xiaomi Mi5 faisant office de point d'accès wifi

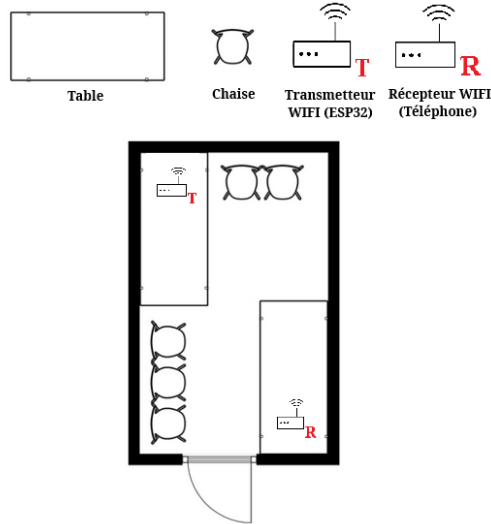


Figure 8: Schéma en vue du dessus de la salle de collecte des données CSI

Pour optimiser le temps pris par la récolte, pour chaque combinaison d'individus, nous avons enregistré en continu pendant un certain moment, puis fragmenté l'enregistrement initial en différents enregistrements de 3 secondes. Le code utilisé pour cette fragmentation est inclus dans la sous-section 6.2.

Nous avons repris la méthode utilisée par le dataset WiMANS[11], à savoir convertir les différentes valeurs des mesures CSI en amplitude [25, 2, 12], ce qui revient à prendre le module de chaque nombre complexe. On a également tenté de garder la structure de WiMANS[11], ainsi, on a la structure suivante :

```
dataset
| - annotation.csv          # labels, annotations (nombre et identites des individus)
| - parsing.py             # le code utilise pour la fragmentation et les amplitudes
| - csi
|   | - 0 (200).txt        # les logs de l'ESP32 bruts avec 0 individus
|   | - 1 B (50).txt       # les logs de l'ESP32 bruts avec 1 individu B
|   | - ...                # au total 15 logs bruts renvoyes par l'ESP32 (fichiers .txt)
| - amplitudes
|   | - 1.npy              # les amplitudes CSI de l'enregistrement "1"
|   | - 2.npy              # les amplitudes CSI de l'enregistrement "2"
|   | - ...                # au total 911 echantillons d'amplitudes (fichier .npy)
```

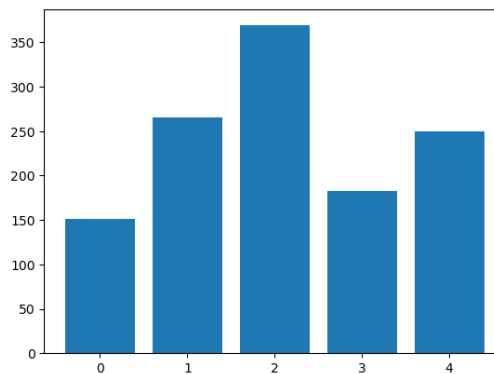


Figure 9: Répartition réelle des échantillons obtenus

Finalement, on a 1218 enregistrements, atteignant ainsi notre objectif.

Néanmoins, certains problèmes nous ont empêchés d'obtenir la répartition souhaitée :

- L'enregistrement des logs avec une combinaison de 2 individus s'est mal enregistré.
- L'enregistrement des logs avec une combinaison de 3 individus a été oublié.

On peut ainsi passer à la prochaine étape, et entraîner un modèle avec nos données.

4.4 Entraînement d'un Réseau de Neurones à Convolution (CNN)

Nous avons finalement opté pour un modèle ResNet [10] (Réseau neuronal résiduel), implémenté en s'aidant grandement d'un notebook trouvé sur GitHub[15]. Notre choix s'est porté vers ce dernier de par sa réputation, sa popularité, ainsi que ses performances, ayant gagné le concours ImageNet[5] 2015.

Le détail de l'implémentation de ce modèle est disponible dans la sous-section 6.3.

Après avoir entraîné le ResNet avec 80% des données à notre disposition, nous l'avons testé avec les 20% de données restants. On a ainsi pu atteindre une précision de 61.89%, ce qui est assez cohérent.

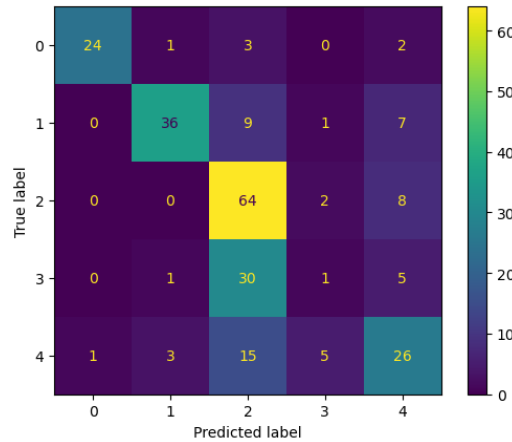


Figure 10: Matrice de confusion du ResNet avec notre dataset

La précision est assez élevée pour montrer que l'analyse des mesures CSI est effectivement pertinente pour la détection humaine, mais reste globalement assez faible.

On peut conjecturer plusieurs raisons à cela :

- Les ESP32 nécessitent une antenne externe pour être vraiment efficace, ce que nous n'avons pas pu installer.
- Les signaux wifi ambiants liés aux téléphones, aux ordinateurs ou à eduroam ont perturbé la collecte.
- Le modèle est très mauvais quand il s'agit de voir qu'il y a 3 individus, ce qui peut être lié à l'oubli de l'un des enregistrements.
- La prédiction de 2 individus se fait souvent alors qu'il y en a 3 ou 4, ce qui peut être lié au fait que c'est avec la classe 2 que nous avons le plus d'exemples.

Néanmoins, le modèle est assez correct quand il s'agit de prédire 0, 1, ou éventuellement 4 individus.

Une fois le modèle entraîné, on peut exécuter la commande `"torch.save(model.state_dict(), "trained_model.pt")"` afin d'enregistrer les différents poids du modèle et pouvoir l'utiliser rapidement en cas de besoin.

Nous avons tiré parti de cette fonctionnalité afin de tenter d'implémenter un algorithme permettant le comptage en temps réel, disponible dans la sous-section 6.4.

Le code, à exécuter dans le répertoire `csi_recv_router`, est normalement fonctionnel, néanmoins dû à des problèmes au niveau du service de réservation de box de Sorbonne Université, nous n'avons pas pu avoir accès à la BDL47 et avons dû nous tourner vers une salle similaire, mais pas identique.

Dans ces conditions, la prédiction était presque systématiquement fausse. Néanmoins, on a noté que quand l'ESP32 ainsi que le point d'accès wifi étaient espacés d'un mètre et demi dans un espace spacieux, l'algorithme prédisait 0 individus s'il n'y avait aucun obstacle entre les deux, et 4 sinon, ce qui est assez cohérent.

5 Conclusion du projet

Afin d’avoir un meilleur contrôle sur toutes les étapes du projet, nous avons mené une campagne expérimentale pour collecter nos propres données CSI. Ceci nous a permis d’avoir une base solide en vue de l’entraînement des modèles personnalisés que nous avons sélectionnés. Ce travail nous a également permis d’explorer de manière rigoureuse la question du seuil minimal de données nécessaires à l’apprentissage d’un modèle performant. Ces résultats ont été obtenus malgré plusieurs contraintes techniques, telles que l’acquisition d’un GPU adapté ou la configuration du Raspberry Pi, qui ont représenté des défis non-négligeables.

Après comparaison des modèles **THAT**, **CNN-2D** et **CLSTM**, le modèle **CNN-2D** avec une taille d’échantillon réduite nous est apparu comme un compromis efficace entre une précision satisfaisante et un temps d’entraînement et de prédiction plus courts. Récolter nos propres données CSI multi-utilisateurs nous a aussi permis de contribuer au travail fait par WiMANS[11], ainsi que mettre en pratique l’utilisation d’un ESP32 pour cette tâche, plus facile à mettre en oeuvre qu’un Raspberry Pi.

Le wifi CSI est une technologie prometteuse, et dispose d’un très bon support sur les cartes ESP32, implémenter un CNN sur ce micro-contrôleur pour compter le nombre de personnes dans une pièce, ou encore utiliser plusieurs micro-contrôleurs afin d’affiner la prédiction sont des pistes à explorer.

6 Annexes

6.1 Installation et utilisation de Nexmon et Nexmon CSI

Les instructions pour l'installation sont issues et adaptées de <https://github.com/seemoo-lab/nexmon>, https://github.com/nexmonster/nexmon_csi/tree/pi-5.4.51-plus et https://github.com/seemoo-lab/nexmon_csi

6.1.1 Installation du Raspberry Pi 4 et modification du firmware

Pour le choix du système d'exploitation, il est nécessaire d'utiliser une version suffisamment ancien, dans ce projet, il a été utilisé la version *2022-09-22-raspbian-bullseye-arm64-lite* récupérable à travers le site suivant : https://downloads.raspberrypi.com/raspbian_lite_arm64/images/raspbian_lite_arm64-2022-09-26/

À la suite de l'écriture de l'image disque et de démarrage sur le Raspberry Pi, il est nécessaire d'exécuter les commandes suivantes :

```
sudo apt update

sudo apt install git libgmp3-dev gawk qpdf bc bison flex libssl-dev make automake \
texinfo libtool-bin tcpdump libncurses5-dev

sudo reboot
```

Attention, il est nécessaire de ne pas exécuter un *apt upgrade* sous peine de mettre à jour le kernel et empêcher la modification du firmware.

Il est ensuite nécessaire d'installer les *Kernel Headers*, pour cela nous utilisons le projet *rpi-source* (<https://github.com/RPi-Distro/rpi-source>).

```
sudo wget https://raw.githubusercontent.com/RPi-Distro/rpi-source/master/rpi-source -O \
/usr/local/bin/rpi-source && sudo chmod +x /usr/local/bin/rpi-source \
&& /usr/local/bin/rpi-source -q --tag-update

rpi-source

sudo reboot
```

Dans le cadre de la version 64bit du système d'exploitation, il est nécessaire d'exécuter les commandes suivantes pour compléter l'environnement avant compilation du firmware.

```
sudo dpkg --add-architecture armhf

sudo apt-get update

sudo apt-get install libc6:armhf libisl23:armhf libmpfr6:armhf libmpc3:armhf libstdc++6:armhf

sudo ln -s /usr/lib/arm-linux-gnueabi/libisl.so.23.0.0 \
/usr/lib/arm-linux-gnueabi/libisl.so.10

sudo ln -s /usr/lib/arm-linux-gnueabi/libmpfr.so.6.1.0 \
/usr/lib/arm-linux-gnueabi/libmpfr.so.4
```

Nous pouvons maintenant passer à la compilation et l'installation du firmware et des outils nécessaires.

```
git clone https://github.com/seemoo-lab/nexmon.git

cd nexmon

source setup_env.sh

make

cd patches/bcm43455c0/7_45_189/nexmon
```

```

git clone https://github.com/seemoo-lab/nexmon_csi.git

cd nexmon_csi

make install-firmware

#Installation de makecsiparams
cd utils/makecsiparams

make

ln -s $PWD/makecsiparams /usr/local/bin/mcp

#Installation de nexutil
cd ../../../../.. #retour à la racine du dossier principal nexmon
cd utilities/nexutil/

make && make install

#Configuration de la persistance du firmware
cd ../.. #retour à la racine du dossier principal nexmon
cd patches/bcm43455c0/7_45_189/nexmon_csi/
cd brcmfmac_5.4.y-nexmon
mv $(modinfo brcmfmac -n) ./brcmfmac.ko.orig
cp ./brcmfmac.ko $(modinfo brcmfmac -n)
depmod -a

```

De manière optionnelle, il peut être utile de retirer *wpa_supplicant* pour obtenir un meilleur contrôle sur l'interface wifi.

```
sudo apt-get remove wpasupplicant
```

6.1.2 Configuration de l'interface d'écoute et utilisation

L'utilisation de la commande *makecsiparams* permet de générer la string de paramètre encodé en base64 qui sera utilisé pour configurer l'extracteur. Cette exemple montre la configuration pour le canal 36, sur la plage des 5GHz, avec une bande passante de 40MHz. Les 2 dernières options ne sont pas à modifier dans le contexte d'un Raspberry Pi puisque celui-ci n'a qu'un coeur et une seule antenne.

```

mcp -c 36/40 -C 1 -N 1
JtgBEQAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA==

```

Il est possible de paramétrer encore plus précisément, notamment en filtrant l'adresse MAC, *makecsiparams*. Les options disponibles sont visibles en exécutant *mcp -h*

Les commandes suivantes configurent l'interface réseau en fonction des paramètres entrés précédemment.

```

ifconfig wlan0 up
nexutil -Iwlan0 -s500 -b -l34 -vJtgBEQAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA==
iw dev wlan0 interface add mon0 type monitor
ip link set mon0 up

```

La capture des paquets UDP forgés par l'extracteur peut se faire grâce à l'utilisation de *tcpdump*.

```

tcpdump -i wlan0 dst port 5500
# Ou pour permettre un enregistrement et traitement ultérieur :
# Enregistrement de 1000 échantillons CSI dans un fichier pcap
tcpdump -i wlan0 dst port 5500 -vv -w output.pcap -c 1000

```

Si besoin, pour vérifier le canal wifi d'écoute et le configurer, il est possible d'utiliser la commande *nexutil -k*.

6.2 Transformation des logs ESP32 en amplitudes .npz (NumPy)

Le code suivant est situé dans un répertoire qui contient un dossier csi avec les différents logs issus de l'ESP32, ainsi qu'un dossier amplitudes vide qui va accueillir les amplitudes des échantillons construits.

Listing 1: parsing.py

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

from os import listdir

#Nombre de signaux par secondes des echantillons
hz = 50

#Nombre de secondes voulues par sequence
s = 3

#Nombre de secondes entre les sequences
jump = 0

#L'emplacement des logs
input_x = "csi/"

#La destination des fichiers .npz d'amplitude
output_x = "amplitudes/"

#La destination du DataFrame associant les echantillons et les differentes
    caracteristiques
output_y = "annotation.csv"

#Nombre de secondes avant enregistrement
s_df = 5

#Nombre de secondes avant fin
f_df = 5

#Fonction pour parser les donnees CSI et les transformer en amplitudes
def csi_to_amp(txt):
    csi = np.array(eval(txt))
    return np.sqrt(csi[:,2]**2 + csi[1:,2]**2)

#Liste des colonnes pour le .csv d'annotations
colonnes = ["#", "compte", "b", "j", "n", "t"]

#Initialisation du DataFrame pandas
label = pd.DataFrame(columns=colonnes)
label.loc[len(label)] = [0, 0, 0, 0, 0, 0]

#On parcourt tous les fichiers de textes presents dans le repertoire
for file_name in listdir(input_x):
    #On ne garde que les fichiers qui nous interessent
    if "txt" == file_name[-3:]:
        print(file_name)
        with open(input_x + file_name, "r", encoding="utf-8") as file:
            #On avance dans le fichier afin d'arriver aux informations qui nous
                interessent
            while line := file.readline():
                if "CSI_DATA" in line:
                    break

            #On enregistre les informations dans un DataFrame pandas afin de pouvoir
                mieux les manipuler
            df = pd.read_csv(file, on_bad_lines='warn')
```

```

#On s'assure que toutes les lignes presentent bien des mesures CSI complets
df = df[df.iloc[:,0] == "CSI_DATA"]
print("Nombre de signaux: " + str(len(df)))

#On retire le debut et la fin de chaque enregistrement, afin d'eviter les
    eventuels participants en trop
df = df.iloc[s_df * hz: len(df) - f_df * hz]

#Calcul des amplitudes
amp = []
for csi in df.iloc[:, -1]:
    amp.append(csi_to_amp(csi))

amp = np.array(amp)

#Detection de la composition a l'aide du nom du fichier
B = "B" in file_name
J = "J" in file_name
N = "N" in file_name
T = "T" in file_name

#Segmentation et enregistrement des amplitudes
for i in range(len(amp)// (hz * (s + jump))):
    n = label["#"].iloc[-1] + 1
    X = amp[i * (hz * (s + jump)): i * (hz * (s + jump)) + hz * s]
    label.loc[len(label)] = [n, B + J + N + T, B, J, N, T]
    X_name = str(n) + ".npy"
    np.save(output_x + X_name, X)

print("Nombre d'echantillons: " + str(i))

label = label.iloc[1:]
label.to_csv("annotation.csv", index=False)

```

6.3 Implémentation du modèle ResNet[10]

Une fois le notebook du ResNet-18 Digit Classifier [15] lancé, on doit tout d'abord configurer les différents paramètres.

- Le batch size à 32, car on n'a que 1000 échantillons.
- Le nombre d'épochs à 12, pour éviter le sur-apprentissage.
- Le nombre de classes à prédire 5, pour chaque nombres d'individus possibles.

Ensuite, on remplace le chargement de MNIST par nos propres données.

En supposant que le notebook est situé dans le même répertoire que le dossier dataset, on va remplacer le code utilisé pour obtenir MNIST par:

```

df = pd.read_csv("./dataset/annotation.csv")

X = np.array([np.load("./dataset/amplitudes/" + str(i) + ".npy") for i in df["#"]
    ])
Y = np.array(df["compte"])

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = TEST_SIZE,
    random_state=0, stratify = Y)

#Le modele prend des tenseurs
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
Y_train_tensor = torch.tensor(Y_train, dtype=torch.long)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
Y_test_tensor = torch.tensor(Y_test, dtype=torch.long)

#Le modele initial prend des valeurs de la forme [1, 28, 28]

```

```

X_train_tensor = X_train_tensor.unsqueeze(1)
X_test_tensor = X_test_tensor.unsqueeze(1)

#Pour mieux organiser les tenseurs
train_dataset = TensorDataset(X_train_tensor, Y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, Y_test_tensor)

#Pour calculer plus rapidement
train_loader = DataLoader(dataset=train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(dataset=test_dataset, batch_size=BATCH_SIZE, shuffle=False)

```

Enfin, il faut modifier la donnée sortant du modèle.

Pour ce faire, on regarde la forme de l'information transitant dans le modèle juste avant le renvoi de sa valeur, en ajoutant un "print(x.shape)" juste après la dernière couche du modèle dans la fonction forward de ce dernier.

```

...
def forward(self, x):
    ...
    x = self.layer4(x)
    print(x.shape)
    x = x.view(x.size(0), -1)
    ...

```

Ce qui nous renvoie ici "torch.Size([32, 512, 5, 2])".

On omettant le 32 représentant le batch size, il faut finalement transformer une information de taille 512 x 5 x 2, en une information de taille 5 (le nombre de classes que l'on cherche). On modifie donc le premier argument de la dernière fonction nn.Linear du modèle, afin qu'on puisse bien avoir la transformation cherchée.

```

...
def __init__(self, block, layers, num_classes, grayscale):
    ...
    self.layer4 = self._make_layer(block, 512, layers[3], stride=2)
    self.avgpool = nn.AvgPool2d(7, stride=1)
    self.fc = nn.Linear(512 * 5 * 2, num_classes)

    for m in self.modules():
        ...

```

6.4 Algorithme de prédiction en temps réel

Listing 2: temp_reel.py

```

import os
import time

import numpy as np

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset

# Architecture
NUM_CLASSES = 5

# Other
DEVICE = "cpu"
GRAYSCALE = True

device = torch.device(DEVICE)

...
-- MODELE RESNET --
...

```

```

model = resnet18(NUM_CLASSES)
model.to(DEVICE)

#Charger le modele qu'on a deja entraine
model.load_state_dict(torch.load('trained_model.pt'))

not_found = True

def csi_to_amp(txt):
    csi = np.array(eval(txt))
    return np.sqrt(csi[:,2]**2 + csi[1:,2]**2)

#On bloucle pour predire en permanence
while True:
    if not_found:
        #On commence par chercher le fichier de logs
        for file_name in os.listdir():
            if file_name[-3:] == ".txt" and not file_name == "CMakeLists.txt":
                not_found = False
                break

        #On transforme les 150 dernieres lignes en amplitudes dans un tenseur
        x_tensor = torch.tensor(np.array([csi_to_amp(line[line.find('"') + 1:-2]) for
            line in open(file_name).readlines()[-151:-1]]), dtype=torch.float32)
        #On fait attention a ce que le modele prennent bien une entree de la forme
        [1, 1, 150, 64]
        x_tensor = x_tensor.unsqueeze(0)
        x_tensor = x_tensor.unsqueeze(0)

        #On fait notre prediction
        model.eval()
        with torch.no_grad():
            _, probas = model(x_tensor)
        predicted_class = int(torch.argmax(probas, dim=1)[0])

        #On l'affiche
        print(predicted_class)

```

6.5 Administratif et outils

Discord : un moyen de communication est indispensable sur un projet collaboratif. Nous avons opté pour Discord étant donné qu'on l'utilise tous régulièrement, et que l'application est disponible sur ordinateur et téléphone facilement, tout en offrant plusieurs fonctionnalités pratiques.

Dépôt Git : afin de pouvoir centraliser nos différentes avancées sur ce projet collaboratif, un dépôt Git est très pratique. Sorbonne Université propose justement une instance Gitlab. L'un des étudiants du groupe ayant vu son compte bloqué sans raison, nous avons dû passer par Madame Fladenmuller afin de débloquent son compte, ce qui n'est toujours pas le cas à ce jour. Nous y avons ainsi créé un repository pour notre projet.

Ordinateurs : ce projet nécessite de pouvoir entraîner des modèles d'Intelligence Artificielle avec un dataset de 53GB. Ceci nécessite des ordinateurs puissants et une capacité de stockage assez conséquente. Nous avons ainsi sollicité la Plateforme Pédagogique et Technique Informatique (PPTI) de Sorbonne Université, qui ont accepté de nous donner l'accès à des ordinateurs adaptés à notre usage, avec des RTX 3080 et 250GB de stockage. L'une des conditions imposées par la PPTI est que l'on doit partager ces ordinateurs avec les autres étudiants, ce qui implique qu'on ne peut pas utiliser ces machines durant les heures de cours.

Overleaf : utilisé massivement dans le monde académique, cet éditeur LaTeX permet de mettre en forme du texte avec une bibliographie, des formules mathématiques ou encore des extensions utiles en informatique. Néanmoins, l'instance en ligne proposée par le projet ne permet qu'un seul collaborateur par projet, de ce fait, on a préféré opter pour une instance hébergée sur un serveur appartenant à Jules.

Raspberry Pi : la 3ème partie du projet nécessite un Raspberry Pi, afin de collecter des données et d'y intégrer un modèle permettant de détecter l'activité humaine nous-même. Ainsi, nous avons pu emprunter un Raspberry Pi 4 auprès du LIP6, par l'intermédiaire de nos encadrants de projet.

ESP32 : ces micro-contrôleurs permettent de collecter les mesures CSI, et ainsi de faire la 3ème partie du projet.

6.6 Réunions

Nous avons pu avoir plusieurs réunions avec nos référents nous permettant d'échanger sur le sujet et notre travail :

- Réunion du 28 janvier : Mise en place du projet et de ses objectifs
- Réunion du 11 février : Présentation de nos débuts de réflexions et proposition de pistes
- Réunion du 18 février : Discussion sur le matériel nécessaire et les problématiques rencontrées. Réception du Raspberry Pi 4.
- Réunion du 11 mars : Discussion sur le rapport intermédiaire et nos avancées
- Réunion du 1 avril : Discussion sur nos avancées et les problèmes rencontrés restants
- Réunion du 8 avril : Discussion sur nos avancées
- Réunion du 30 avril : Discussion sur nos avancées et sur la structure du rapport

Bibliographie

- [1] A. Aileen, A. Suwardi, and F. Prawiranata. WiFi Signal Strength Degradation Over Different Building Materials. *Engineering, Mathematics and Computer Science (EMACS) Journal*, 3:109–113, Oct. 2021.
- [2] Z. Chen, L. Zhang, C. Jiang, Z. Cao, and W. Cui. Wifi csi based passive human activity recognition using attention based blstm. *IEEE Transactions on Mobile Computing*, 18(11):2714–2724, Nov. 2019. Publisher Copyright: © 2018 IEEE.
- [3] E. Contributors. Applications based on wi-fi csi (channel state information), such as indoor positioning, human detection. https://github.com/espressif/esp-csi/tree/master/examples/get-started/csi_recv_router, 2022.
- [4] M. Contributors. Openmmlab’s next generation video understanding toolbox and benchmark. <https://github.com/open-mmlab/mmdetection>, 2020.
- [5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [6] Espressif. Esp-idf. <https://idf.espressif.com/>.
- [7] S. Ghosh, O. Vinyals, B. Strope, S. Roy, T. Dean, and L. Heck. Contextual lstm (clstm) models for large scale nlp tasks, 2016.
- [8] F. Gringoli, M. Schulz, J. Link, and M. Hollick. Free your csi: A channel state information extraction platform for modern wi-fi chipsets. In *Proceedings of the 13th International Workshop on Wireless Network Testbeds, Experimental Evaluation Characterization, WiNTECH ’19*, page 21–28, 2019.
- [9] L. Guo, L. Wang, C. Lin, J. Liu, B. Lu, J. Fang, Z. Liu, Z. Shan, J. Yang, and S. Guo. Wiar: A public dataset for wifi-based activity recognition. *IEEE Access*, 7:154935–154945, 2019.
- [10] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition, 2015.
- [11] S. Huang, K. Li, D. You, Y. Chen, A. Lin, S. Liu, X. Li, and J. A. McCann. WiMANS: A Benchmark Dataset for WiFi-based Multi-user Activity Sensing, Mar. 2024. arXiv:2402.09430 [eess].
- [12] B. Li, W. Cui, W. Wang, L. Zhang, Z. Chen, and M. Wu. Two-stream convolution augmented transformer for human activity recognition. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(1):286–293, May 2021.
- [13] H. Mo and S. Kim. A deep learning-based human identification system with wi-fi csi data augmentation. *IEEE Access*, 9:91913–91920, 2021.
- [14] P. F. Moshiri, M. Nabati, R. Shahbazian, and S. A. Ghorashi. Csi-based human activity recognition using convolutional neural networks. In *2021 11th International Conference on Computer Engineering and Knowledge (ICCKE)*, page 7–12, Mashhad, Iran, Islamic Republic of, Oct. 2021. IEEE.
- [15] S. Raschka. Deep learning models. <https://github.com/rasbt/deeplearning-models>.
- [16] G. Retscher. Fundamental Concepts and Evolution of Wi-Fi User Localization: An Overview Based on Different Case Studies. *Sensors (Basel, Switzerland)*, 20(18):5121, Sept. 2020.
- [17] E. Salomon, L. H. Botler, K. Diwold, C. A. Boano, and K. Römer. Poster: Comparison of Channel State Information driven and RSSI-based WiFi Distance Estimation. In *Proceedings of the 2021 International Conference on Embedded Wireless Systems and Networks, EWSN ’21*, pages 173–174, USA, Apr. 2021. Junction Publishing.
- [18] M. Schulz, D. Wegemer, and M. Hollick. Nexmon: The c-based firmware patching framework, 2017.
- [19] C. Tan, X. Feng, J. Long, and L. Geng. Forecast-clstm: A new convolutional lstm network for cloudage nowcasting. In *2018 IEEE Visual Communications and Image Processing (VCIP)*, pages 1–4, 2018.
- [20] F. Wang, J. Feng, Y. Zhao, X. Zhang, S. Zhang, and J. Han. Joint activity recognition and indoor localization with wifi fingerprints. *IEEE Access*, 7:80058–80068, 2019.
- [21] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick. Detectron2. <https://github.com/facebookresearch/detectron2>, 2019.

- [22] J. Yang, X. Chen, D. Wang, H. Zou, C. X. Lu, S. Sun, and L. Xie. Sensefi: A library and benchmark on deep-learning-empowered wifi human sensing, 2023.
- [23] Z. Yang, Y. Zhang, G. Chi, and G. Zhang. Hands-on Wireless Sensing with Wi-Fi: A Tutorial, June 2022. arXiv:2206.09532 [eess].
- [24] Z. Yang, Z. Zhou, and Y. Liu. From RSSI to CSI: Indoor localization via channel response. *ACM Comput. Surv.*, 46(2):25:1–25:32, Dec. 2013.
- [25] S. Yousefi, H. Narui, S. Dayal, S. Ermon, and S. Valaee. A survey on behavior recognition using wifi channel state information. *IEEE Communications Magazine*, 55(10):98–104, 2017.